



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

N° de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 5 de septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.1 Dibujar las iniciales de sus nombres cada letra de un color diferente.

Al igual que en la primer práctica, elegí mostrar las iniciales de mis dos primeros nombres y mi apellido (AMZ), para que todas fueran distintas entre sí. También me apoyé del ejercicio de clase para utilizar cuadrados rojos, verdes y azules.

Dentro de la función *main()* agregué los siguientes bloques de código:

El primer bloque es dónde dibujo 4 cuadrados rojos y mediante transformaciones los acomodo para formar la letra A.

```
347 // Primer letra (A)
348
349 // Cuadrado rojo
350 model = glm::mat4(1.0f);
351 model = glm::translate(model, glm::vec3(-0.8f, -0.05f, -4.0f));
352 model = glm::scale(model, glm::vec3(0.1f, 0.80f, 1.0f));
353 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
354 meshColorList[3]->RenderMeshColor();
355
356 // Cuadrado rojo
357 model = glm::mat4(1.0f);
358 model = glm::translate(model, glm::vec3(-0.5f, -0.05f, -4.0f));
359 model = glm::scale(model, glm::vec3(0.1f, 0.80f, 1.0f));
360 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
361 meshColorList[3]->RenderMeshColor();
362
363 // Cuadrado rojo
364 model = glm::mat4(1.0f);
365 model = glm::translate(model, glm::vec3(-0.65f, 0.4f, -4.0f));
366 model = glm::scale(model, glm::vec3(0.4f, 0.13f, 1.0f));
367 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
368 meshColorList[3]->RenderMeshColor();
369
370 // Cuadrado rojo
371 model = glm::mat4(1.0f);
372 model = glm::translate(model, glm::vec3(-0.65f, 0.0f, -4.0f));
373 model = glm::scale(model, glm::vec3(0.4f, 0.13f, 1.0f));
374 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
375 meshColorList[3]->RenderMeshColor();
```

El segundo bloque es dónde dibujo 4 cuadrados verdes y mediante transformaciones los acomodo para formar la letra M.

```
377 // Segunda letra (M)
378
379 // Cuadrado verde
380 model = glm::mat4(1.0f);
381 model = glm::translate(model, glm::vec3(-0.20f, 0.0f, -4.0f));
382 model = glm::scale(model, glm::vec3(0.1f, 0.9f, 1.0f));
383 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
384 meshColorList[2]->RenderMeshColor();
385
386 // Cuadrado verde
387 model = glm::mat4(1.0f);
388 model = glm::translate(model, glm::vec3(0.20f, 0.0f, -4.0f));
389 model = glm::scale(model, glm::vec3(0.1f, 0.9f, 1.0f));
390 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
391 meshColorList[2]->RenderMeshColor();
392
393 // Cuadrado verde
394 model = glm::mat4(1.0f);
395 model = glm::translate(model, glm::vec3(-0.075f, 0.31f, -4.0f));
396 model = glm::rotate(model,
397     glm::radians(45.0f),
398     glm::vec3(0.0f, 0.0f, 1.0f));
399 model = glm::scale(model, glm::vec3(0.1f, 0.3f, 1.0f));
400 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
401 meshColorList[2]->RenderMeshColor();
402
403 // Cuadrado verde
404 model = glm::mat4(1.0f);
405 model = glm::translate(model, glm::vec3(0.075f, 0.31f, -4.0f));
406 model = glm::rotate(model,
407     glm::radians(135.0f),
408     glm::vec3(0.0f, 0.0f, 1.0f));
409 model = glm::scale(model, glm::vec3(0.1f, 0.3f, 1.0f));
410 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
411 meshColorList[2]->RenderMeshColor();
412
```

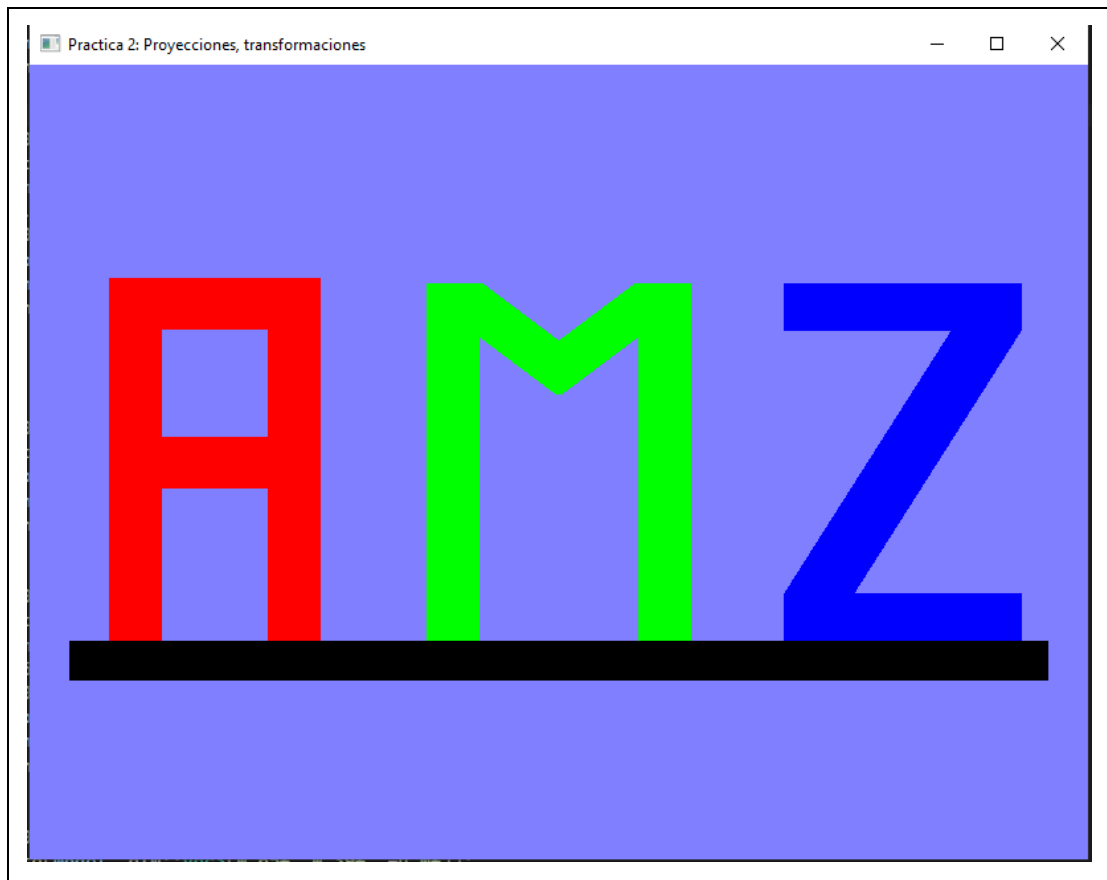
Finalmente, el tercer bloque es dónde dibujo 3 cuadrados azules y mediante transformaciones los acomodo para formar la letra Z.

```

413 // Tercera letra (Z)
414
415 // Cuadrado azul
416 model = glm::mat4(1.0f);
417 model = glm::translate(model, glm::vec3(0.65f, -0.39f, -4.0f));
418 model = glm::scale(model, glm::vec3(0.45f, 0.12f, 1.0f));
419 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
420 meshColorList[8]->RenderMeshColor();
421
422 // Cuadrado azul
423 model = glm::mat4(1.0f);
424 model = glm::translate(model, glm::vec3(0.65f, 0.0f, -4.0f));
425 model = glm::rotate(model,
426     glm::radians(64.5f),
427     glm::vec3(0.0f, 0.0f, 1.0f));
428 model = glm::scale(model, glm::vec3(0.79f, 0.12f, 1.0f));
429 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
430 meshColorList[8]->RenderMeshColor();
431
432 // Cuadrado azul
433 model = glm::mat4(1.0f);
434 model = glm::translate(model, glm::vec3(0.65f, 0.39f, -4.0f));
435 model = glm::scale(model, glm::vec3(0.45f, 0.12f, 1.0f));
436 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
437 meshColorList[8]->RenderMeshColor();
438

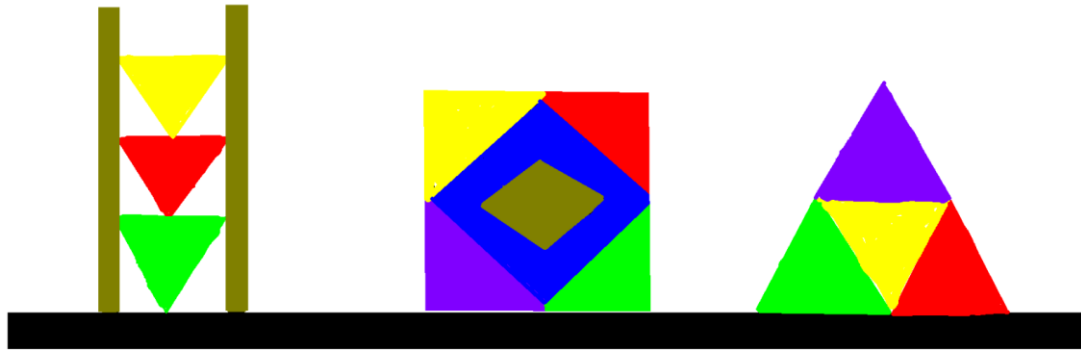
```

Adjunto captura de pantalla de mis iniciales (AMZ):



1.2 Generar el dibujo de las 3 figuras de clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear *shaders* diferentes de los colores: verde, azul, café y magenta adicionales al *shaderrojo*.

Color diferente a blanco, negro y los de las figuras



Para esto primero hice los *vertex shaders* correspondientes a cada color (verde, azul, amarillo, café, morado y negro) y utilicé el *fragment shader* de *shadercolor* ya que ese no cambiaba.

```
28 // Vertex Shaders de colores
29 static const char* vShaderRojo = "shaders/shaderrojo.vert";
30 static const char* vShaderVerde = "shaders/shaderverde.vert";
31 static const char* vShaderAzul = "shaders/shaderazul.vert";
32 static const char* vShaderAmarillo = "shaders/shaderamarillo.vert";
33 static const char* vShaderCafe = "shaders/shadercafe.vert";
34 static const char* vShaderMorado = "shaders/shadermorado.vert";
35 static const char* vShaderNegro = "shaders/shadernegro.vert";
```

Después modifiqué la función *CreateShaders()* para crear los *shaders* de los nuevos colores.

```

304 void CreateShaders()
305 {
306     // Shader rojo
307     Shader* shaderRojo = new Shader();
308     shaderRojo->CreateFromFiles(vShaderRojo, fShaderColor);
309     shaderList.push_back(*shaderRojo); // índice 0
310
311     // Shader verde
312     Shader* shaderVerde = new Shader();
313     shaderVerde->CreateFromFiles(vShaderVerde, fShaderColor);
314     shaderList.push_back(*shaderVerde); // índice 1
315
316     // Shader azul
317     Shader* shaderAzul = new Shader();
318     shaderAzul->CreateFromFiles(vShaderAzul, fShaderColor);
319     shaderList.push_back(*shaderAzul); // índice 2
320
321     // Shader amarillo
322     Shader* shaderAmarillo = new Shader();
323     shaderAmarillo->CreateFromFiles(vShaderAmarillo, fShaderColor);
324     shaderList.push_back(*shaderAmarillo); // índice 3
325
326     // Shader café
327     Shader* shaderCafe = new Shader();
328     shaderCafe->CreateFromFiles(vShaderCafe, fShaderColor);
329     shaderList.push_back(*shaderCafe); // índice 4
330
331     // Shader morado
332     Shader* shaderMorado = new Shader();
333     shaderMorado->CreateFromFiles(vShaderMorado, fShaderColor);
334     shaderList.push_back(*shaderMorado); // índice 5
335
336     // Shader negro
337     Shader* shaderNegro = new Shader();
338     shaderNegro->CreateFromFiles(vShaderNegro, fShaderColor);
339     shaderList.push_back(*shaderNegro); // índice 6
340 }

```

Posteriormente comencé a armar las figuras haciendo uso del *shaderList* y del *meshList*, dónde el shader indicaba el color y el mesh el tipo de figura (pirámide triangular, pirámide cuadrangular o cubo).

De acuerdo a los índices del *shaderList*, los índices de colores son los siguientes:

rojo → 0, verde → 1, azul → 2, amarillo → 3, café → 4, morado → 5,
negro → 6

Y para *meshList*, los índices de las figuras son:

pirámide triangular → 0, cubo → 1, pirámide cuadrangular → 2

Los bloques de código modificados para las 3 figuras son:

```
496 // Figura 1
497 // Barra café
498 shaderList[4].useShader();
499 uniformModel = shaderList[4].getModelLocation();
500 uniformProjection = shaderList[4].getProjectLocation();
501 model = glm::mat4(1.0f);
502 model = glm::translate(model, glm::vec3(-1.8f, 0.13f, -3.0f));
503 model = glm::scale(model, glm::vec3(0.1f, 1.8f, 0.3f));
504 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
505 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
506 meshList[1] -> RenderMesh();
507
508 // Barra café
509 shaderList[4].useShader();
510 uniformModel = shaderList[4].getModelLocation();
511 uniformProjection = shaderList[4].getProjectLocation();
512 model = glm::mat4(1.0f);
513 model = glm::translate(model, glm::vec3(-1.2f, 0.13f, -3.0f));
514 model = glm::scale(model, glm::vec3(0.1f, 1.8f, 0.3f));
515 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
516 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
517 meshList[1] -> RenderMesh();
518
519 // Pirámide amarilla
520 shaderList[3].useShader();
521 uniformModel = shaderList[3].getModelLocation();
522 uniformProjection = shaderList[3].getProjectLocation();
523 model = glm::mat4(1.0f);
524 model = glm::translate(model, glm::vec3(-1.5f, 0.5f, -3.0f));
525 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
526 model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
527 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
528 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
529 meshList[0] -> RenderMesh();
530
531 // Pirámide roja
532 shaderList[0].useShader();
533 uniformModel = shaderList[0].getModelLocation();
534 uniformProjection = shaderList[0].getProjectLocation();
535 model = glm::mat4(1.0f);
536 model = glm::translate(model, glm::vec3(-1.5f, 0.0f, -3.0f));
537 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
538 model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
539 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
540 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
541 meshList[0] -> RenderMesh();
542
543 // Pirámide verde
544 shaderList[1].useShader();
545 uniformModel = shaderList[1].getModelLocation();
546 uniformProjection = shaderList[1].getProjectLocation();
547 model = glm::mat4(1.0f);
548 model = glm::translate(model, glm::vec3(-1.5f, -0.5f, -3.0f));
549 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
550 model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
551 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
552 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
553 meshList[0] -> RenderMesh();
```

```

556 // Figura 2
557 // Cubo café (rombo)
558 shaderList[4].useShader();
559 uniformModel = shaderList[4].getModelLocation();
560 uniformProjection = shaderList[4].getProjectLocation();
561 model = glm::mat4(1.0f);
562 model = glm::translate(model, glm::vec3(-0.18f, -0.24f, -3.0f));
563 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
564 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
565 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
566 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
567 meshList[1]->RenderMesh();
568
569 // Cubo azul (rombo)
570 shaderList[2].useShader();
571 uniformModel = shaderList[2].getModelLocation();
572 uniformProjection = shaderList[2].getProjectLocation();
573 model = glm::mat4(1.0f);
574 model = glm::translate(model, glm::vec3(-0.18f, -0.24f, -3.0f));
575 model = glm::scale(model, glm::vec3(0.64f, 0.64f, 0.3f));
576 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
577 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
578 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
579 meshList[1]->RenderMesh();
580
581 // Pirámide roja
582 shaderList[0].useShader();
583 uniformModel = shaderList[0].getModelLocation();
584 uniformProjection = shaderList[0].getProjectLocation();
585 model = glm::mat4(1.0f);
586 model = glm::translate(model, glm::vec3(0.05f, -0.02f, -3.0f));
587 model = glm::scale(model, glm::vec3(0.50f, 0.50f, 0.3f));
588 model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
589 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
590 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
591 meshList[2]->RenderMesh();
592
593 // Pirámide verde
594 shaderList[1].useShader();
595 uniformModel = shaderList[1].getModelLocation();
596 uniformProjection = shaderList[1].getProjectLocation();
597 model = glm::mat4(1.0f);
598 model = glm::translate(model, glm::vec3(0.05f, -0.52f, -3.0f));
599 model = glm::scale(model, glm::vec3(0.50f, 0.50f, 0.3f));
600 model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
601 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
602 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
603 meshList[2]->RenderMesh();
604
605 // Pirámide amarilla
606 shaderList[3].useShader();
607 uniformModel = shaderList[3].getModelLocation();
608 uniformProjection = shaderList[3].getProjectLocation();
609 model = glm::mat4(1.0f);
610 model = glm::translate(model, glm::vec3(-0.45f, -0.02f, -3.0f));
611 model = glm::scale(model, glm::vec3(0.50f, 0.50f, 0.3f));
612 model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
613 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
614 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
615 meshList[2]->RenderMesh();
616
617 // Pirámide morada
618 shaderList[5].useShader();
619 uniformModel = shaderList[5].getModelLocation();
620 uniformProjection = shaderList[5].getProjectLocation();
621 model = glm::mat4(1.0f);
622 model = glm::translate(model, glm::vec3(-0.45f, -0.52f, -3.0f));
623 model = glm::scale(model, glm::vec3(0.50f, 0.50f, 0.3f));
624 model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
625 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
626 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
627 meshList[2]->RenderMesh();

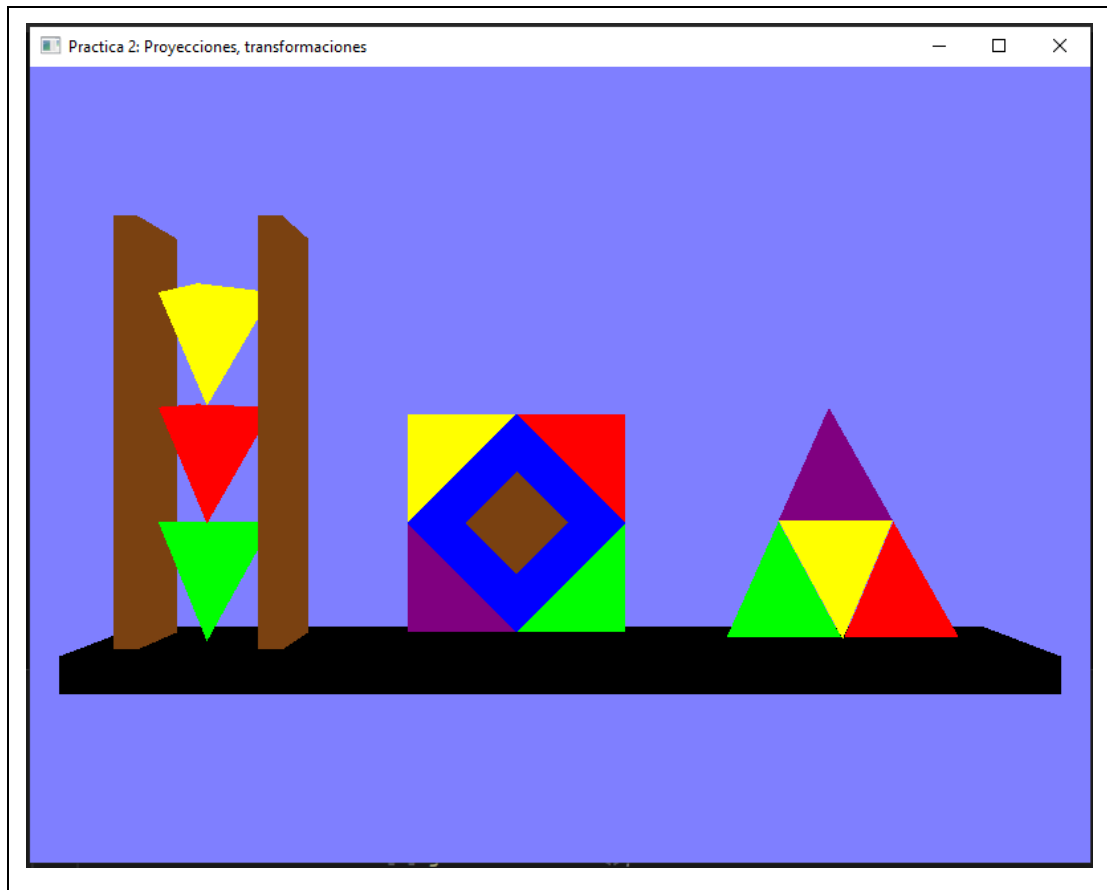
```

```

629 // Figura 3
630 // Pirámide amarilla
631 shaderList[3].useShader();
632 uniformModel = shaderList[3].getModelLocation();
633 uniformProjection = shaderList[3].getProjectLocation();
634 model = glm::mat4(1.0f);
635 model = glm::translate(model, glm::vec3(1.2f, -0.49f, -3.0f));
636 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
637 model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
638 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
639 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
640 meshList[0]->RenderMesh();
641
642 // Pirámide morada
643 shaderList[5].useShader();
644 uniformModel = shaderList[5].getModelLocation();
645 uniformProjection = shaderList[5].getProjectLocation();
646 model = glm::mat4(1.0f);
647 model = glm::translate(model, glm::vec3(1.2f, 0.005f, -3.0f));
648 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
649 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
650 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
651 meshList[0]->RenderMesh();
652
653 // Pirámide verde
654 shaderList[1].useShader();
655 uniformModel = shaderList[1].getModelLocation();
656 uniformProjection = shaderList[1].getProjectLocation();
657 model = glm::mat4(1.0f);
658 model = glm::translate(model, glm::vec3(0.975f, -0.503f, -3.0f));
659 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
660 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
661 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
662 meshList[0]->RenderMesh();
663
664 // Pirámide roja
665 shaderList[0].useShader();
666 uniformModel = shaderList[0].getModelLocation();
667 uniformProjection = shaderList[0].getProjectLocation();
668 model = glm::mat4(1.0f);
669 model = glm::translate(model, glm::vec3(1.487f, -0.503f, -3.0f));
670 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.3f));
671 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
672 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
673 meshList[0]->RenderMesh();
674
675
676 // Figura 4
677 // Barra negra
678 shaderList[6].useShader();
679 uniformModel = shaderList[6].getModelLocation();
680 uniformProjection = shaderList[6].getProjectLocation();
681 model = glm::mat4(1.0f);
682 model = glm::translate(model, glm::vec3(0.0f, -0.84f, -3.0f));
683 model = glm::scale(model, glm::vec3(4.0f, 0.15f, 0.5f));
684 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
685 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
686 meshList[1]->RenderMesh();

```

Adjunto captura de pantalla de la ejecución del programa:



2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

En esta ocasión no hubo ningún problema durante la realización de la práctica.

3. Conclusión:

- a. Los ejercicios del reporte:** Me agrado mucho el hecho de que “repetimos” los ejercicios de la práctica 1 pero con los nuevos conceptos visto para esta práctica, así pude visualizar de mejor manera las diferencias y beneficios de usar transformaciones geométricas y el uso de diversos shaders.
- b. Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado a transformaciones geométricas.
- c. Conclusión.** Esta práctica me sirvió mucho para visualizar como se modifican diversas figuras en el espacio después de aplicarle ciertas transformaciones geométricas (*rotate*, *scale*, *translate*) con sus parámetros correspondientes, así como observar que los puntos de vista (ortogonal y con perspectiva) afectan el cómo se ven las figuras al momento de hacer la ejecución.

Referencias bibliográficas.

Narf the Mouse. (2011, September 1). *What is the correct order to multiply scale, rotation and translation matrices for a proper world matrix?* Game Development Stack Exchange.
<https://gamedev.stackexchange.com/questions/16719/what-is-the-correct-order-to-multiply-scale-rotation-and-translation-matrices-f>